LAPORAN PRAKTIKUM PEMROGRAMAN WEB
MODUL 6: GENERATE DATA OTOMATIS & AUTHENTICATION



Disusun Oleh:

Nama: Andhika Yuridho

NIM: 707012500059

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

UNIVERSITAS TELKOM

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan

Adapun tujuan yang ingin dicapai pada praktikum Modul 6 ini adalah sebagai berikut:

1. Memahami konsep authentication, session, hashing password, dan middleware pada Laravel.

2. Memasang Laravel Breeze dan menggunakan fitur register, login, serta logout yang dibuat otomatis.

3. Memahami alur register, login, logout, dan middleware auth yang berjalan di balik layar.

4. Membuat akun pengguna dummy dengan seeder dan Faker.

5. Menguji halaman yang dilindungi middleware auth serta akun hasil seeder pada aplikasi yang berjalan.

2. Alat dan Bahan

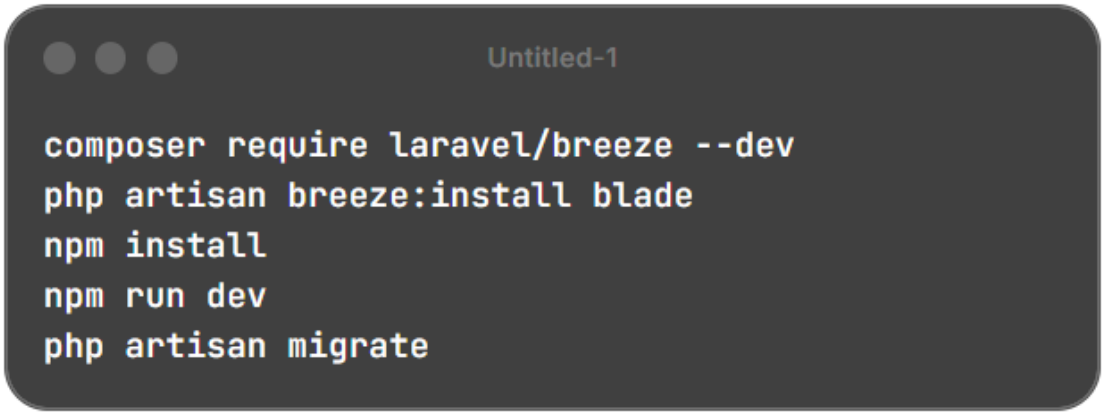
Praktikum ini dikerjakan menggunakan satu buah laptop dengan prosesor Intel Core i7-13650HX dan sistem operasi Windows 11. Perangkat lunak yang digunakan meliputi Laragon sebagai lingkungan pengembangan lokal, PHP versi 8.3.30, Composer versi 2.10.3, Laravel Framework versi 13.37.0, Laravel Breeze versi 2.4.2, Node.js versi 24.14.1, NPM versi 11.15.0, SQLite versi 3.40.0 sebagai database, Visual Studio Code sebagai editor, Git, dan Google Chrome sebagai browser. Project yang dipakai adalah project Laravel baru di folder praktikum6-jurnal.

3. Langkah-Langkah Praktikum

Langkah-langkah praktikum ini diuraikan sesuai materi 6.4.5 sampai 6.4.8, mulai dari instalasi Laravel Breeze, pengujian route authentication, penelusuran alur kerja authentication, sampai pembuatan user dummy dengan seeder dan Faker. Project yang dipakai adalah project Laravel baru, sehingga route dan view dari modul sebelumnya tidak perlu diselamatkan.

3.1 Instalasi Laravel Breeze

Laravel Breeze dipasang sebagai dependency pengembangan melalui Composer. Perintah composer require laravel/breeze --dev mengunduh package Breeze ke folder vendor dan menambahkan laravel/breeze pada bagian require-dev di composer.json. Setelah package tersedia, perintah php artisan breeze:install blade membuat route, controller, view, dan file pendukung authentication dengan tampilan Blade.



Perintah breeze:install blade juga menjalankan npm install dan npm run build secara otomatis. Saya tetap menjalankan npm install satu kali dan hasilnya up to date. Vite dijalankan dengan npm run dev di terminal terpisah agar perubahan CSS dan JavaScript langsung ter-compile selama pengujian. Terminal kedua dipakai untuk php artisan serve.

Proses instalasi mengganti isi routes/web.php dan beberapa view bawaan. Karena project ini baru, tidak ada route atau view dari praktikum sebelumnya yang hilang.

Migration dijalankan dengan php artisan migrate. Pada project ini perintah tersebut menampilkan pesan 'Nothing to migrate' karena tabel users, password_reset_tokens, sessions, dan tabel bawaan Laravel lainnya sudah terbentuk. Pesan itu bukan galat, hanya menandakan tidak ada migration baru yang perlu dijalankan.

Breeze menghasilkan tiga bagian utama. Route authentication ada di routes/auth.php, view ada di resources/views/auth, dan controller ada di app/Http/Controllers/Auth. Ketiganya membentuk pola MVC yang sama seperti modul sebelumnya, dengan route sebagai alamat URL, controller sebagai pemroses data, dan view sebagai tampilan.

3.2 Routing dan Pengujian Authentication

Breeze mengubah routes/web.php menjadi seperti berikut:

Untitled-1

```php
<?php

use App\Http\Controllers\ProfileController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
});

require __DIR__.'/auth.php';
```

Baris require __DIR__.'/auth.php'; memuat seluruh route authentication dari file terpisah. Route /dashboard memakai middleware auth dan verified, sehingga halaman itu hanya bisa dibuka oleh pengguna yang sudah login. Middleware verified memeriksa verifikasi email, tetapi model User bawaan belum memakai MustVerifyEmail, jadi pengguna baru tetap bisa masuk ke dashboard.

File routes/auth.php berisi route register, login, logout, dan fitur authentication lain. Route register dan login berada di dalam grup middleware guest, sedangkan logout berada di grup auth.

Untitled-1

```php
<?php

use App\Http\Controllers\Auth\AuthenticatedSessionController;
use App\Http\Controllers\Auth\ConfirmablePasswordController;
use App\Http\Controllers\Auth\EmailVerificationNotificationController;
use App\Http\Controllers\Auth\EmailVerificationPromptController;
use App\Http\Controllers\Auth\NewPasswordController;
use App\Http\Controllers\Auth\PasswordController;
use App\Http\Controllers\Auth\PasswordResetLinkController;
use App\Http\Controllers\Auth\RegisteredUserController;
use App\Http\Controllers\Auth\VerifyEmailController;
use Illuminate\Support\Facades\Route;

Route::middleware('guest')->group(function () {
    Route::get('register', [RegisteredUserController::class, 'create'])
        ->name('register');

    Route::post('register', [RegisteredUserController::class, 'store']);

    Route::get('login', [AuthenticatedSessionController::class, 'create'])
        ->name('login');

    Route::post('login', [AuthenticatedSessionController::class, 'store']);

    Route::get('forgot-password', [PasswordResetLinkController::class, 'create'])
        ->name('password.request');

    Route::post('forgot-password', [PasswordResetLinkController::class, 'store'])
        ->name('password.email');

    Route::get('reset-password/{token}', [NewPasswordController::class, 'create'])
        ->name('password.reset');

    Route::post('reset-password', [NewPasswordController::class, 'store'])
        ->name('password.store');
});

Route::middleware('auth')->group(function () {
    Route::get('verify-email', EmailVerificationPromptController::class)
        ->name('verification.notice');

    Route::get('verify-email/{id}/{hash}', VerifyEmailController::class)
        ->middleware(['signed', 'throttle:6,1'])
        ->name('verification.verify');

    Route::post('email/verification-notification',
[EmailVerificationNotificationController::class, 'store'])
        ->middleware('throttle:6,1')
        ->name('verification.send');

    Route::get('confirm-password', [ConfirmablePasswordController::class, 'show'])
        ->name('password.confirm');

    Route::post('confirm-password', [ConfirmablePasswordController::class, 'store']);

    Route::put('password', [PasswordController::class, 'update'])->name('password.update');

    Route::post('logout', [AuthenticatedSessionController::class, 'destroy'])
        ->name('logout');
});
```

Nama route login wajib ada karena middleware auth mengarahkan pengguna yang belum login ke route dengan nama tersebut. Route logout memakai method POST supaya proses keluar tidak bisa dipicu hanya dengan membuka sebuah tautan.

Proses register ditangani RegisteredUserController. Method create() menampilkan view auth.register, sedangkan method store() memvalidasi input, membuat user baru, dan langsung melakukan login.

Untitled-1

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Auth\Events\Registered;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\Rules;
use Illuminate\Validation\ValidationException;
use Illuminate\View\View;

class RegisteredUserController extends Controller
{
    /**
     * Display the registration view.
     */
    public function create(): View
    {
        return view('auth.register');
    }

    /**
     * Handle an incoming registration request.
     *
     * @throws ValidationException
     */
    public function store(Request $request): RedirectResponse
    {
        $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'lowercase', 'email', 'max:255',
'unique:'.User::class],
            'password' => ['required', 'confirmed', Rules\Password::defaults()],
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
        ]);

        event(new Registered($user));

        Auth::login($user);

        return redirect(route('dashboard', absolute: false));
    }
}
```

Validasi pada method store() mewajibkan nama, email yang unik, dan password yang dikonfirmasi. Password di-hash dengan Hash::make() sebelum disimpan, lalu event

Registered dipicu dan Auth::login() membuat session untuk user tersebut. Setelah itu pengguna diarahkan ke dashboard.

Proses login ditangani LoginRequest dan AuthenticatedSessionController. LoginRequest memvalidasi email dan password, lalu memanggil Auth::attempt() dengan kredensial tersebut. File LoginRequest saya tampilkan dalam dua bagian agar kodenya tetap terbaca.

Untitled-1

```php
<?php

namespace App\Http\Requests\Auth;

use Illuminate\Auth\Events\Lockout;
use Illuminate\Contracts\Validation\ValidationRule;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\Str;
use Illuminate\Validation\ValidationException;

class LoginRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return [
            'email' => ['required', 'string', 'email'],
            'password' => ['required', 'string'],
        ];
    }
```

Bagian pertama berisi deklarasi class, method authorize(), dan method rules() yang menentukan aturan validasi email dan password.

Untitled-1

```
    /**
     * Attempt to authenticate the request's credentials.
     *
     * @throws ValidationException
     */
    public function authenticate(): void
    {
        $this->ensureIsNotRateLimited();

        if (! Auth::attempt($this->only('email', 'password'), $this->boolean('remember'))) {
            RateLimiter::hit($this->throttleKey());

            throw ValidationException::withMessages([
                'email' => trans('auth.failed'),
            ]);
        }

        RateLimiter::clear($this->throttleKey());
    }

    /**
     * Ensure the login request is not rate limited.
     *
     * @throws ValidationException
     */
    public function ensureIsNotRateLimited(): void
    {
        if (! RateLimiter::tooManyAttempts($this->throttleKey(), 5)) {
            return;
        }

        event(new Lockout($this));

        $seconds = RateLimiter::availableIn($this->throttleKey());

        throw ValidationException::withMessages([
            'email' => trans('auth.throttle', [
                'seconds' => $seconds,
                'minutes' => ceil($seconds / 60),
            ]),
        ]);
    }

    /**
     * Get the rate limiting throttle key for the request.
     */
    public function throttleKey(): string
    {
        return Str::transliterate(Str::lower($this->string('email')).'|'.$this->ip());
    }
}
```

Bagian kedua berisi method authenticate(), ensureIsNotRateLimited(), dan throttleKey(). Method authenticate() memanggil Auth::attempt() untuk mencari user berdasarkan email dan mencocokkan password dengan hash yang tersimpan. Jika cocok, rate limiter dibersihkan dan

proses berlanjut. Jika gagal, ValidationException dilempar dengan pesan auth.failed yang muncul di halaman login.

AuthenticatedSessionController menyelesaikan proses login dan logout. Setelah autentikasi berhasil, session diregenerasi untuk mencegah session fixation, lalu pengguna diarahkan ke halaman yang sebelumnya ingin dibuka atau ke dashboard.

Untitled-1

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

class AuthenticatedSessionController extends Controller
{
    /**
     * Display the login view.
     */
    public function create(): View
    {
        return view('auth.login');
    }

    /**
     * Handle an incoming authentication request.
     */
    public function store(LoginRequest $request): RedirectResponse
    {
        $request->authenticate();

        $request->session()->regenerate();

        return redirect()->intended(route('dashboard', absolute: false));
    }

    /**
     * Destroy an authenticated session.
     */
    public function destroy(Request $request): RedirectResponse
    {
        Auth::guard('web')->logout();

        $request->session()->invalidate();

        $request->session()->regenerateToken();

        return redirect('/');
    }
}
```

Method destroy() menjalankan Auth::guard('web')->logout(), mengosongkan session dengan invalidate(), dan mengganti token CSRF dengan regenerateToken(). Pengguna kemudian diarahkan ke halaman utama.

Pengujian dilakukan dengan php artisan serve di alamat http://127.0.0.1:8000. Halaman /dashboard yang dibuka sebelum login diarahkan ke /login. Setelah register, pengguna langsung masuk ke dashboard. Tombol logout mengembalikan pengguna ke halaman utama, dan login dengan password salah menampilkan pesan galat. Hasil pengujian ditunjukkan pada Gambar 1 sampai Gambar 7.

3.3 Alur Kerja Authentication pada Breeze

Alur register dimulai ketika pengguna membuka GET /register. Breeze menampilkan resources/views/auth/register.blade.php. Form dikirim ke POST /register, lalu RegisteredUserController memvalidasi nama, email, dan password. Jika valid, user baru dibuat dengan password yang sudah di-hash, event Registered dipicu, dan Auth::login() langsung membuat session. Pengguna diarahkan ke /dashboard.

Alur login dimulai dari GET /login yang menampilkan resources/views/auth/login.blade.php. Form dikirim ke POST /login dan diterima AuthenticatedSessionController. LoginRequest memeriksa email serta password, lalu Auth::attempt() mencocokkan keduanya dengan data di tabel users. Jika berhasil, Laravel menyimpan status login pada session dan pengguna diarahkan ke /dashboard. Jika gagal, pengguna kembali ke halaman login dengan pesan galat.

Middleware auth bekerja sebelum request sampai ke controller. Saat pengguna membuka /dashboard, middleware memeriksa apakah session login masih ada. Jika ada, request diteruskan ke view dashboard. Jika tidak ada, pengguna diarahkan ke route login. Middleware verified berjalan setelah auth dan memeriksa verifikasi email, tetapi fitur itu belum aktif pada model User bawaan.

Logout dijalankan dari tombol di halaman dashboard yang mengirim POST /logout. AuthenticatedSessionController@destroy menghapus status login, mengosongkan session, dan mengganti token CSRF. Setelah itu pengguna diarahkan ke halaman utama.

Register dan login sama-sama membuat session, middleware auth memeriksa session sebelum halaman dibuka, dan logout menghapus session tersebut.

3.4 Membuat User Dummy dengan Seeder dan Faker

Seeder dipakai untuk mengisi tabel users dengan akun contoh, sehingga pengujian login tidak perlu mendaftar satu per satu melalui halaman register. File seeder dibuat dengan perintah berikut.



Perintah make:seeder menghasilkan file database/seeders/UserSeeder.php. Isi file tersebut saya ganti dengan kode berikut.

Untitled-1

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;
use Faker\Factory as Faker;

class UserSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $faker = Faker::create('id_ID');

        for ($i = 0; $i < 5; $i++) {
            DB::table('users')->insert([
                'name' => $faker->name,
                'email' => $faker->unique()->safeEmail,
                'password' => Hash::make('password'),
                'created_at' => now(),
                'updated_at' => now(),
            ]);
        }

        // Akun admin manual
        DB::table('users')->insert([
            'name' => 'Admin',
            'email' => 'admin@example.com',
            'password' => Hash::make('admin123'),
            'created_at' => now(),
            'updated_at' => now(),
        ]);
    }
}
```

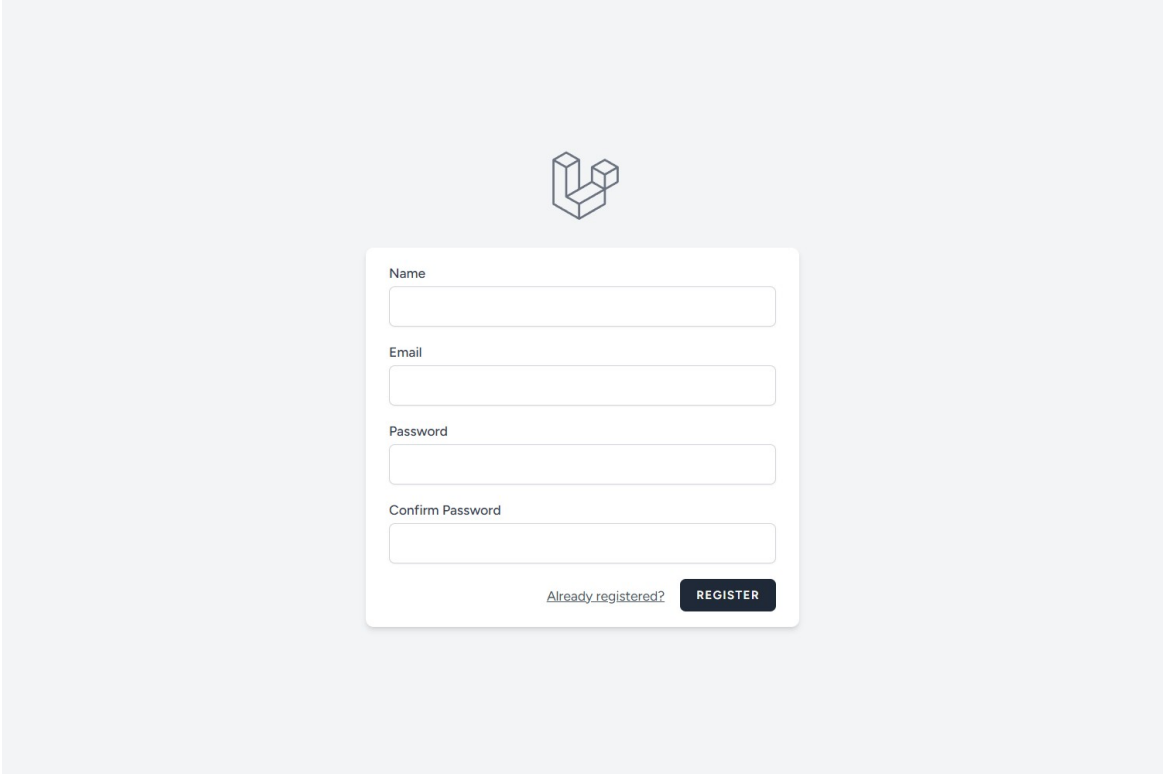
Faker::create('id_ID') membuat data dengan format Indonesia. Perulangan for menjalankan pengisian lima kali, $faker->unique()->safeEmail menghasilkan email acak yang tidak kembar, dan Hash::make('password') menyimpan password dummy dalam bentuk hash. Data dimasukkan ke tabel users melalui DB::table('users')->insert([...]) dengan kolom created_at dan updated_at diisi now().

Selain lima akun dummy, seeder menambahkan satu akun admin tetap dengan email admin@example.com dan password admin123. Password akun admin juga di-hash sebelum disimpan.

Seeder dijalankan dengan php artisan db:seed --class=UserSeeder. Hasilnya tabel users berisi enam akun, yaitu lima akun dummy dan satu akun admin. Saya juga mendaftarkan satu akun melalui halaman register untuk pengujian, sehingga saat laporan ini disusun tabel users berisi tujuh baris. Login dengan akun dummy dan akun admin sama-sama berhasil masuk ke dashboard. Kolom password pada seluruh baris berisi hash bcrypt yang diawali $2y$12$ dengan panjang 60 karakter.

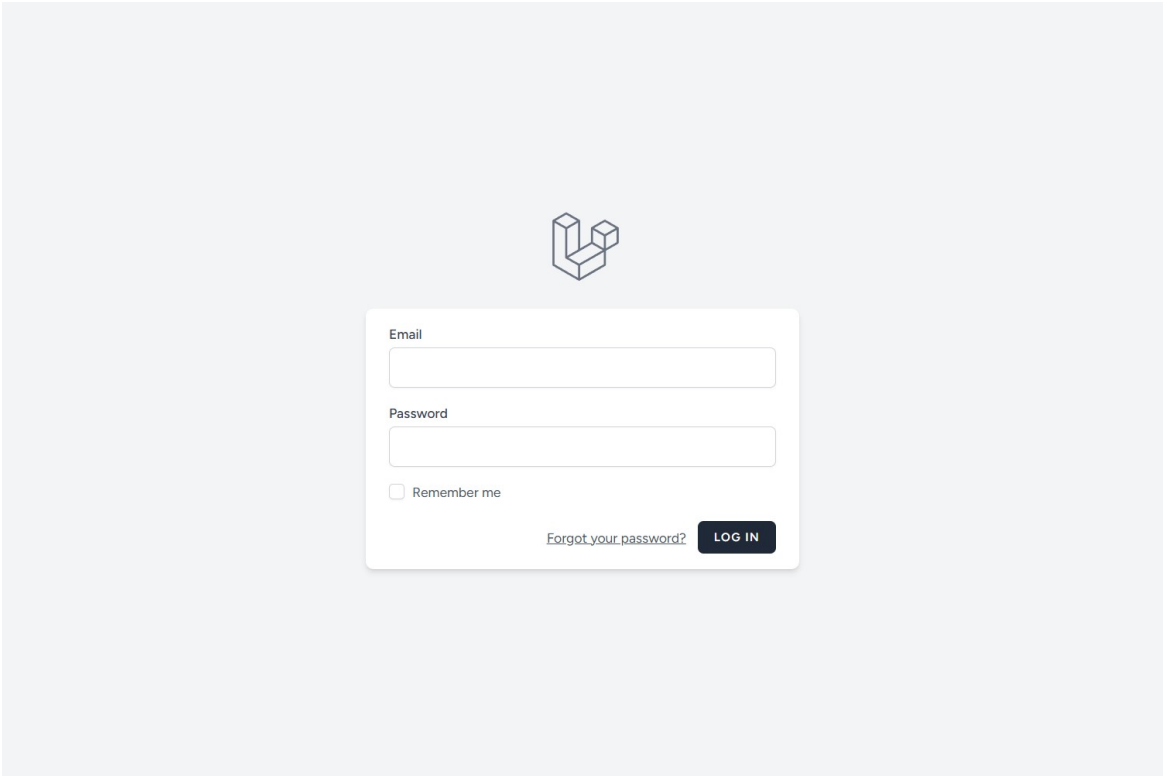
4. Hasil dan Dokumentasi Program

Berikut adalah tangkapan layar aplikasi yang berjalan melalui php artisan serve pada alamat http://127.0.0.1:8000. Karena project ini memakai SQLite, data tabel users ditampilkan melalui query sqlite3, bukan phpMyAdmin.



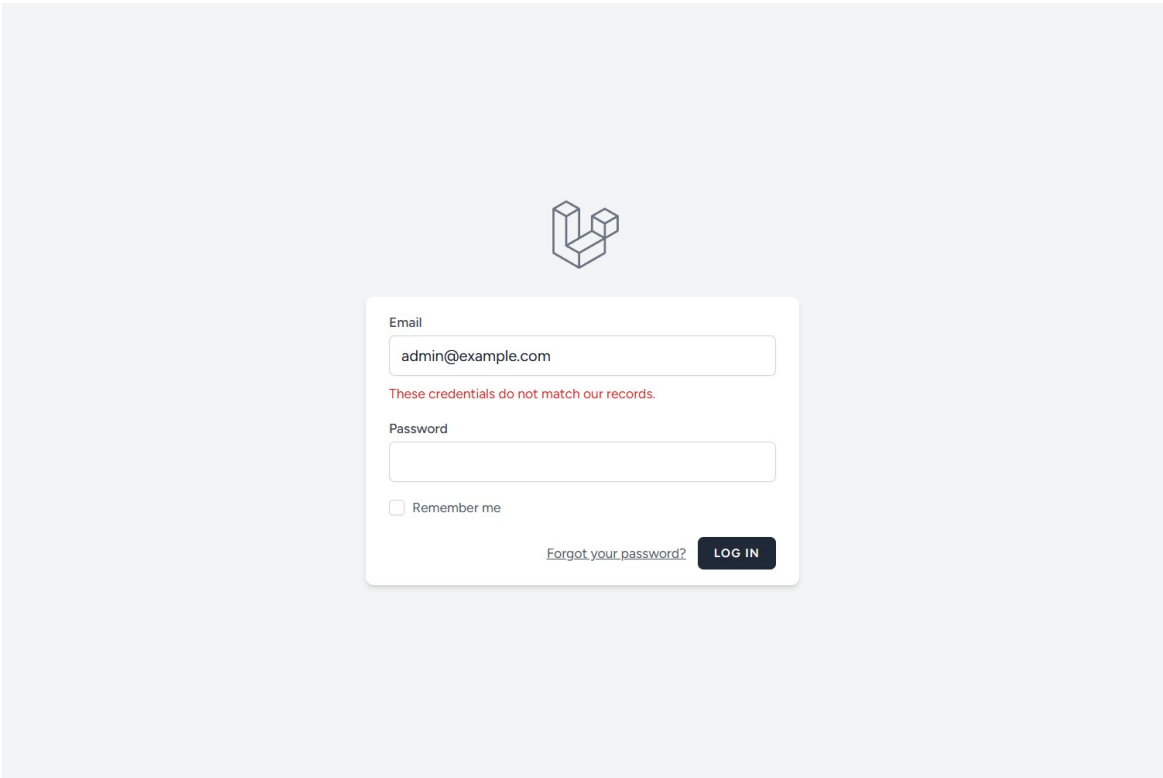
Gambar 1. Halaman register pada alamat /register

Gambar 1 menampilkan form register dengan kolom Name, Email, Password, dan Confirm Password. Form ini dirender oleh resources/views/auth/register.blade.php dan dikirim ke route register dengan method POST.



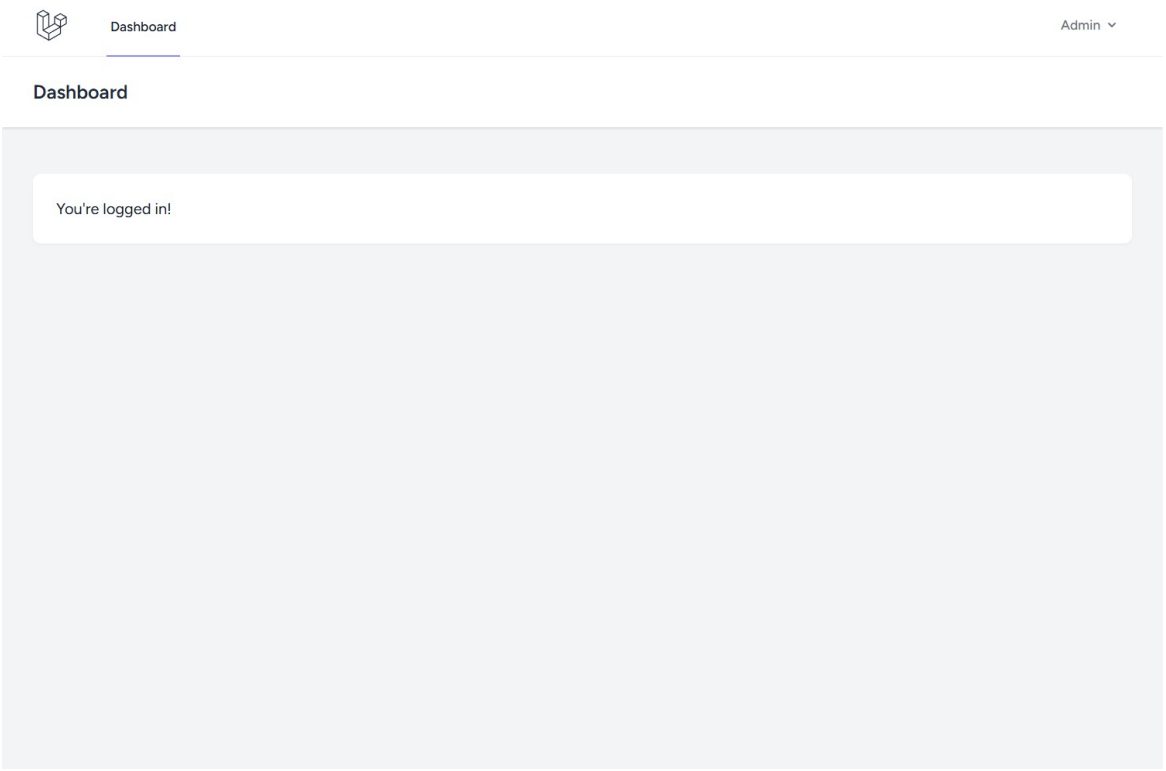
Gambar 2. Halaman login pada alamat /login

Gambar 2 menampilkan form login dengan kolom Email dan Password, checkbox Remember me, tautan Forgot your password, dan tombol LOG IN.



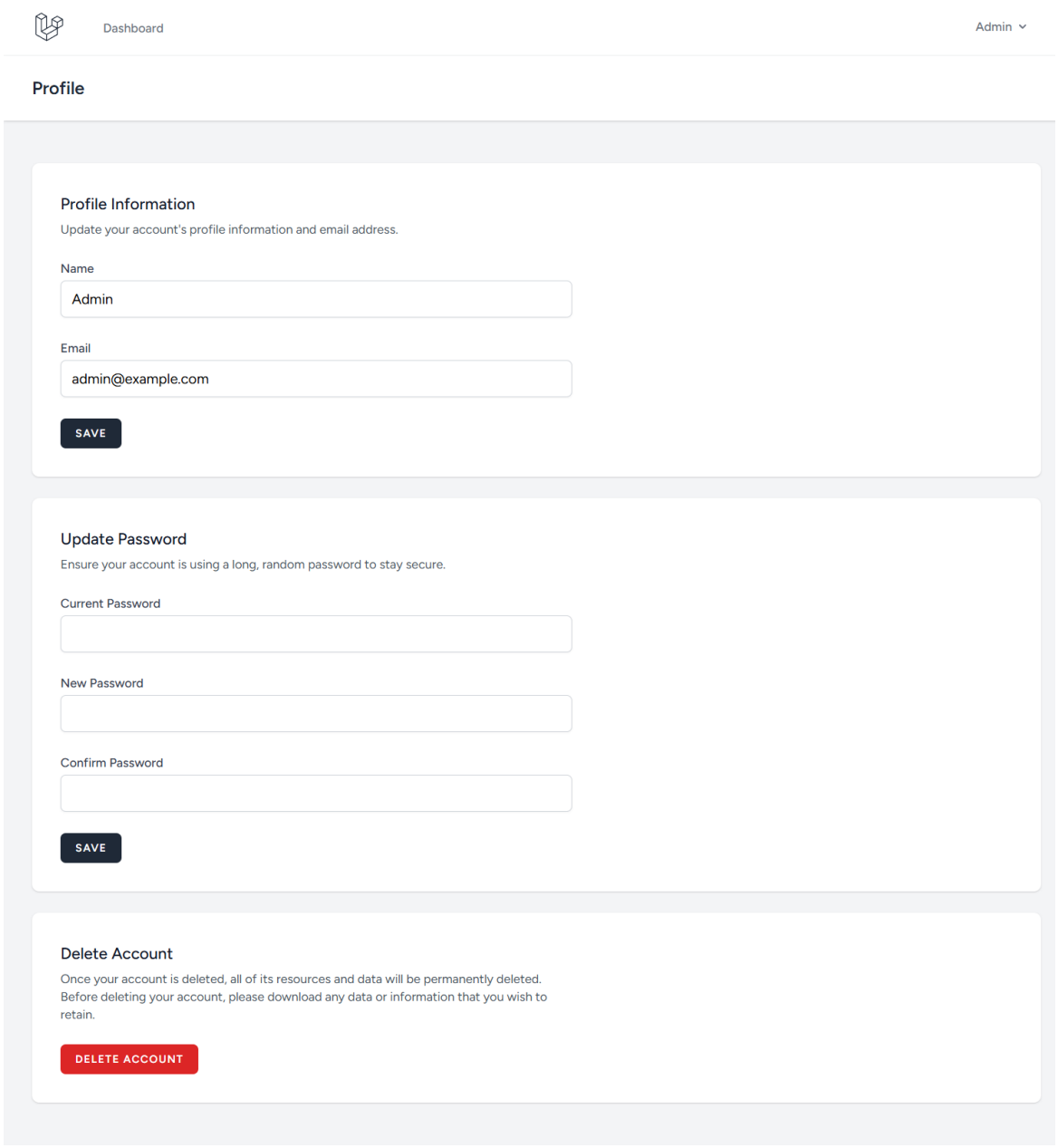
Gambar 3. Pesan galat ketika login menggunakan password yang salah

Gambar 3 memperlihatkan pesan 'These credentials do not match our records.' di bawah kolom Email. Pesan ini muncul dari ValidationException pada LoginRequest setelah Auth::attempt() gagal mencocokkan password.



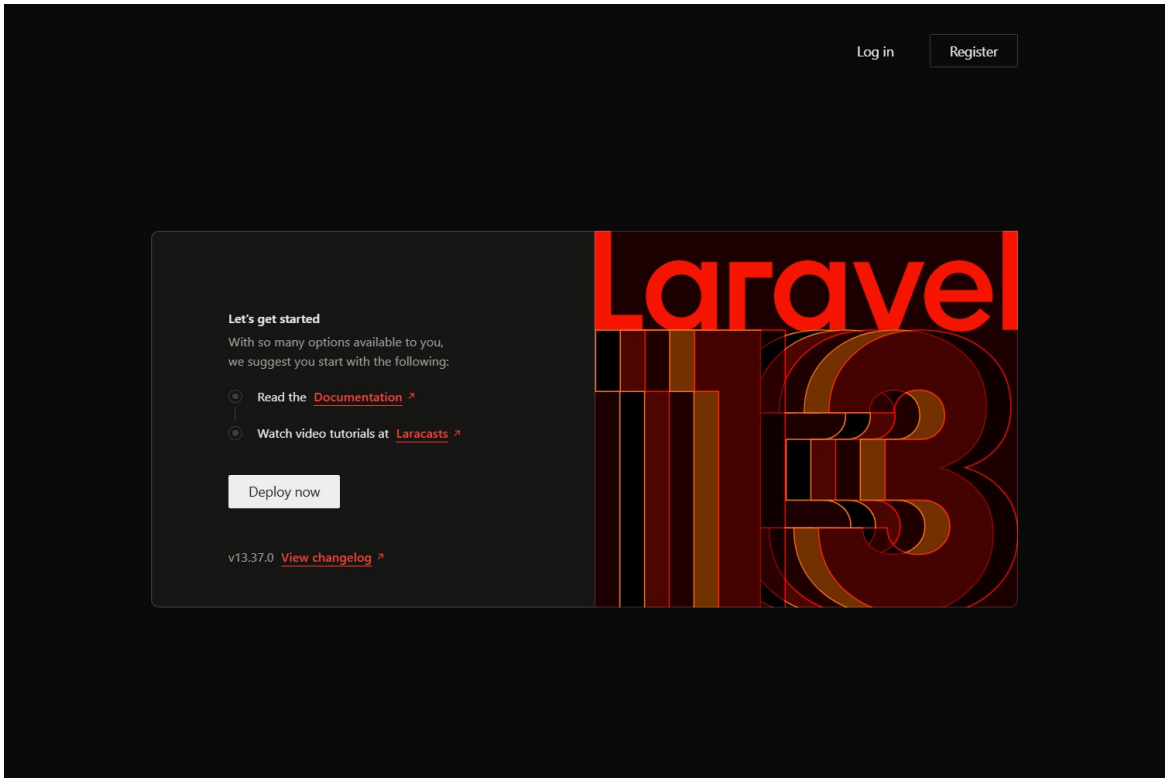
Gambar 4. Halaman dashboard setelah login

Gambar 4 menampilkan halaman dashboard dengan nama pengguna Admin di pojok kanan atas. Halaman ini hanya bisa dibuka setelah login karena route /dashboard memakai middleware auth.



Gambar 5. Halaman profile pengguna yang sedang login

Gambar 5 menampilkan halaman profile Breeze dengan data Admin dan admin@example.com. Halaman ini juga berada di dalam grup middleware auth.



Gambar 6. Halaman utama dengan menu Log in dan Register sebelum login

Gambar 6 menampilkan halaman utama ketika pengguna belum login. Menu Log in dan Register muncul di pojok kanan atas, sedangkan menu Dashboard, Profile, dan Logout baru muncul setelah pengguna login.

Untitled-1

```
            id = 1
          name = Jati Dongoran
         email = humaira.simanjuntak@example.net
      password = $2y$12$lvJQzoAAgkcoeZRb.qcqOevG.x7JPMfi7XLGLL9aFg1TaECZmW5Be
password_length = 60

            id = 2
          name = Respati Hardana Lazuardi M.M.
         email = nasyiah.novi@example.net
      password = $2y$12$QFdcZ7Uo9/feNJ8PY5QQXuCOxV9/hZEeZDNeKJEXThdRMaYwPpDxG
password_length = 60

            id = 6
          name = Admin
         email = admin@example.com
      password = $2y$12$klsM3TfrGqWDFWbKiZ3/FeAn6w98n5/cB38grqwc.hJ73kKMLR4T6
password_length = 60
```

Gambar 7. Data tabel users pada database SQLite

Gambar 7 menampilkan sebagian isi tabel users. Kolom password berisi hash bcrypt yang diawali $2y$12$ dengan panjang 60 karakter, bukan password asli. Akun admin@example.com dan akun dummy hasil seeder dapat dipakai untuk login.

5. Analisis dan Pembahasan

Perbedaan paling terasa antara aplikasi sebelum dan sesudah memakai authentication ada pada siapa yang boleh membuka halaman. Pada project Modul 4, halaman daftar bisa dibuka oleh siapa saja yang mengetahui alamatnya. Setelah route /dashboard dan /profile dilindungi middleware auth, halaman tersebut hanya bisa diakses setelah login. Pengguna yang belum login diarahkan ke /login, lalu dikembalikan ke halaman tujuan setelah berhasil masuk.

Session menjadi penghubung antara proses login dan request berikutnya. Laravel menyimpan ID session pada cookie di browser dan data session di server. Pada project ini SESSION_DRIVER diisi database, sehingga data session tersimpan di tabel sessions. Setelah login berhasil, session()->regenerate() membuat ID baru agar ID lama tidak bisa dipakai ulang. Middleware auth membaca session tersebut untuk menentukan apakah pengguna sudah login.

Password disimpan dalam bentuk hash bcrypt, bukan teks asli. Hash::make() menghasilkan nilai yang diawali $2y$12$ dengan panjang 60 karakter, dan nilai itu tidak bisa dikembalikan menjadi password asli. Saat login, Auth::attempt() mencocokkan password yang diketik dengan hash di database. Proses ini juga dipakai oleh akun hasil seeder, karena seeder memanggil Hash::make() sebelum menyimpan data.

Middleware auth dan nama route login saling bergantung. Middleware mengarahkan pengguna yang belum login ke route bernama login. Tanpa nama tersebut, Laravel akan memunculkan galat Route [login] not defined. Pada routes/auth.php, route GET /login memang diberi nama login, sehingga pengalihan berjalan.

Laravel Breeze membuat route, controller, dan view authentication secara otomatis. Kode yang dihasilkan cukup ringkas, tetapi proses di baliknya tetap bisa dibaca dari RegisteredUserController, LoginRequest, dan AuthenticatedSessionController. Pembuatan authentication secara manual akan memerlukan controller dan route sendiri, sedangkan Breeze menyediakan semuanya dalam satu perintah. Pada praktikum ini saya memakai Breeze, jadi fokus pengerjaan ada pada pemahaman alur dan pengujian.

Seeder dengan Faker mengisi tabel users dengan lima akun dummy dan satu akun admin. Faker::create('id_ID') menghasilkan nama dan email dengan format Indonesia, sedangkan

unique()->safeEmail() mencegah email kembar yang akan melanggar aturan unique pada kolom email. Akun admin dibuat dengan email dan password tetap agar mudah dipakai untuk pengujian. Seluruh password, termasuk password akun admin, disimpan dalam bentuk hash.

Ada satu penyesuaian yang saya lakukan terhadap modul. Modul menggunakan MySQL dan phpMyAdmin, sedangkan project ini memakai SQLite. Karena itu, perintah php artisan migrate menampilkan 'Nothing to migrate' dan pengecekan tabel users dilakukan melalui query sqlite3, bukan phpMyAdmin. Perbedaan ini tidak mengubah alur authentication, karena Laravel memakai Eloquent dan Query Builder yang sama untuk kedua database.

Perintah npm install berjalan normal setelah Breeze dipasang. Hasilnya up to date karena Breeze sudah mengunduh dependency JavaScript yang dibutuhkan. Vite kemudian dijalankan dengan npm run dev dan tetap aktif selama pengujian.

6. Kesimpulan

Berdasarkan praktikum Modul 6 ini, saya memahami authentication sebagai proses memastikan identitas pengguna sebelum halaman tertentu dibuka. Laravel Breeze berhasil dipasang dan menghasilkan route register, login, dan logout, serta controller dan view yang lengkap. Pengujian di browser menunjukkan bahwa halaman dashboard tidak bisa dibuka sebelum login, register langsung membuat session, password salah menghasilkan pesan galat, dan logout mengakhiri session.

Session, hashing password, dan middleware auth saling melengkapi. Session menyimpan status login, hashing menjaga password tetap aman di database, dan middleware memeriksa session sebelum request sampai ke halaman yang dilindungi. Seeder dengan Faker berhasil membuat lima akun dummy dan satu akun admin dengan password yang sudah di-hash. Dengan demikian, fitur authentication pada project praktikum6-jurnal berjalan sesuai materi 6.4.5 sampai 6.4.8.

7. Link Repository Github

https://github.com/dhodhoo/pabw-andhika-yuridho/tree/main/pabw-6/praktikum6-jurnal